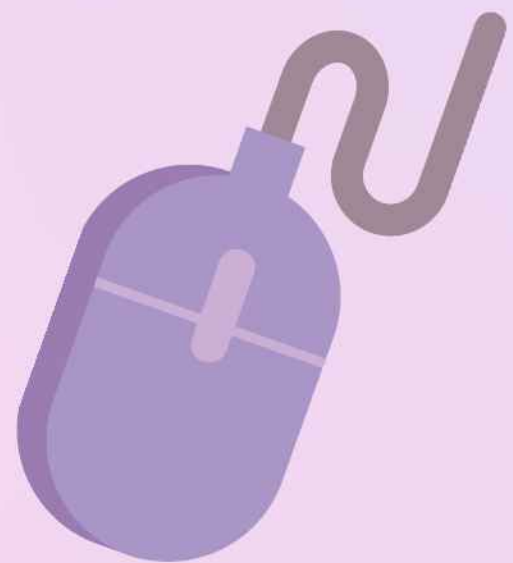




DATA FLOW DIAGRAMMING



Genrhey Guhao Barba



DATA FLOW DIAGRAMMING



What Is Data Flow Diagramming?

Data Flow Diagramming is a way of visually representing how data moves through an information system or business process. It helps us see where data comes from, how it is processed, where it is stored, and where it goes.

DFDs make complex systems easier to understand and are useful in software engineering, systems analysis, process improvement, business management, and agile software development.



Examples:



Data Flow – Shows the movement of data from one part of a system to another.



Information System – A system where data is received, processed, stored, and delivered.



Business Process – A series of activities that uses data to accomplish a task.



DATA FLOW DIAGRAM



How Does a DFD Show Data Movement?

A Data Flow Diagram (DFD) shows how data moves through a system. It focuses on where the data comes from, how the system processes it, where it is stored, and where it goes.

A DFD focuses on data movement, while other diagrams may focus on activities, structure, or system behavior.



Examples:

-  **Source** – The person, organization, or system that provides data to a process.
-  **Process** – A system activity that changes or handles incoming data.
-  **Destination** – The place or entity that receives the resulting data.



DFD VS. FLOWCHART



What Is the Difference Between a DFD and a Flowchart?

A DFD focuses on the movement of data through a system, while a Flowchart focuses on the step-by-step sequence of activities or logic in a process.

A flowchart can show decisions and the order of tasks, while a DFD focuses more on data movement, processes, storage, and external entities.



Examples:



DFD – Shows how data moves within a system.



Flowchart – Shows the sequence of activities or logic.



Process Logic – Shows how activities are performed in order, including decisions.



PURPOSE OF DATA FLOW DIAGRAMS



Why Do We Use DFDs?

DFDs help simplify complex systems into visuals that are easier to understand. They can also help identify system requirements by showing relationships between data, processes, and external entities.

DFDs can support communication among analysts, designers, developers, and stakeholders, and can also help identify inefficient or unnecessary data handling.



Examples:



System Understanding – Makes a complex system easier to visualize and understand.



Communication – Provides a common diagram that different people can discuss.



Process Improvement – Helps identify unnecessary or inefficient data handling.



COMPONENTS OF A DFD



What Are the Main Components of a DFD?

A DFD uses specific symbols to represent different parts of a system. The main components shown in the lesson are processes, data stores, external entities, and data flows.

These symbols work together to show how information moves through the system.



Examples:

-  **Process** – Represents an activity that handles or transforms data.
-  **Data Store** – Represents a place where information is kept.
-  **External Entity** – Represents a person or system outside the system boundary.



LOGICAL DFD



What Is a Logical DFD?

A Logical DFD provides a high-level view of the data flows needed to perform business or system processes.

It focuses on what the system does and what data is needed, without describing the specific hardware, software, or technical implementation.



Examples:



Business Process – Shows the business activity being performed.



Data Requirement – Shows what information is needed by the process.



Data Movement – Shows how information moves between processes and other parts of the system.



PHYSICAL DFD



What Is a Physical DFD?

A Physical DFD visualizes how a system or process is actually implemented. It can include the hardware, software, files, procedures, technologies, and operations required to perform the process.



Examples:



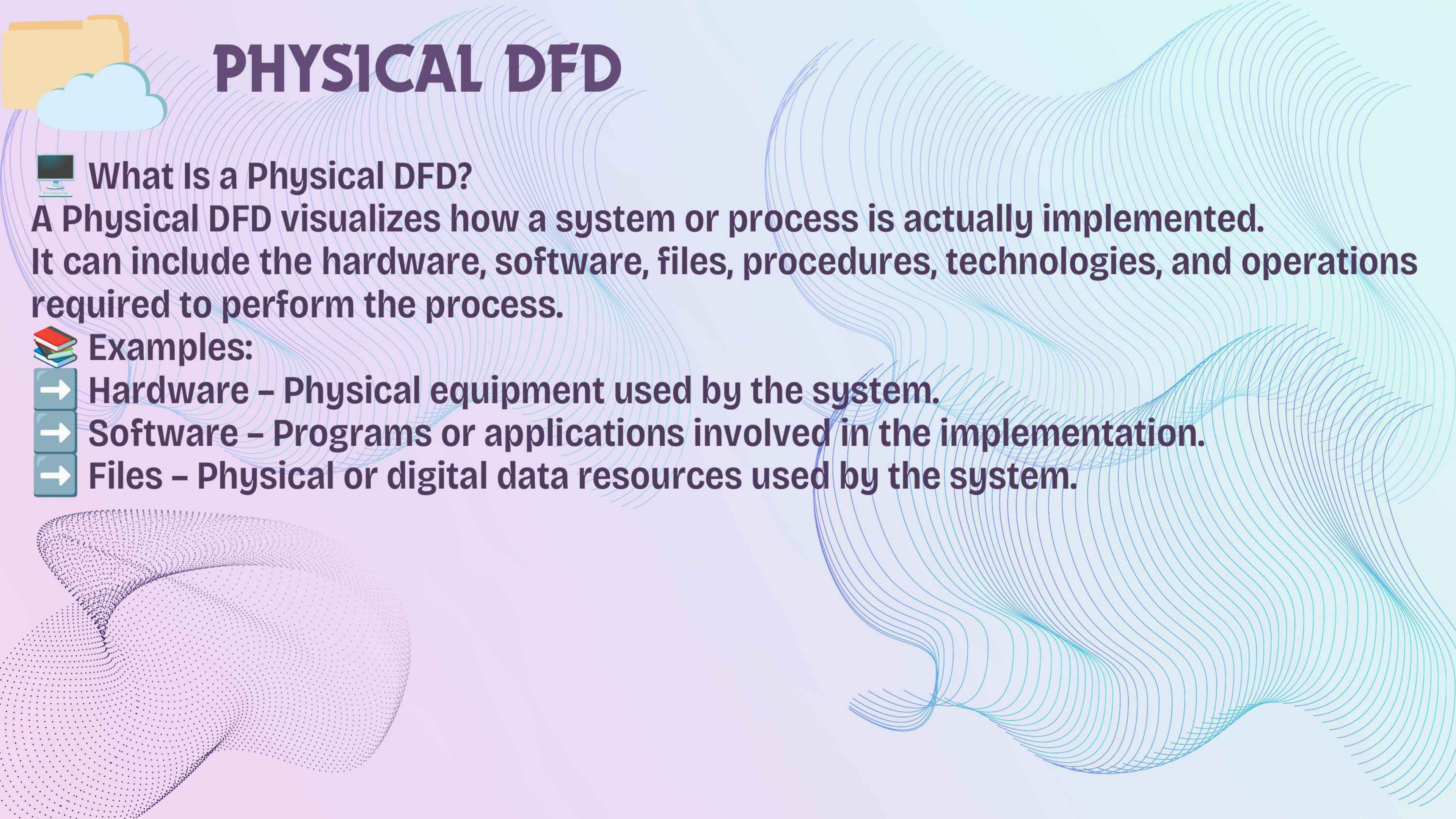
Hardware – Physical equipment used by the system.



Software – Programs or applications involved in the implementation.



Files – Physical or digital data resources used by the system.





LEVELS OF A DFD



How Are DFDs Organized Into Levels?

DFDs can be created at different levels of detail. Level 0 gives the big-picture view, while higher levels break processes into more detailed parts.

The lesson identifies Level 0, Level 1, Level 2, and Level 3 for increasingly detailed representations.



Examples:



Level 0 – Shows the entire system as one main process with its inputs and outputs.



Level 1 – Breaks the main process into smaller subprocesses.



Level 2 – Provides greater detail about subprocesses, data flows, and data stores.



RULES FOR DATA FLOW DIAGRAMS



How Should DFD Elements Be Named?

DFD elements should use clear and meaningful labels so readers can easily understand what the data or process represents.

Data flows should identify the data being moved, processes should describe the transformation, and data stores should identify the stored information.



Examples:



Data Flow Label – Uses brief descriptive text that identifies the data being moved.



Process Label – Uses a brief verb phrase describing what the process does.



Data Store Label – Uses a noun or noun phrase describing the stored data.



DFD CONNECTION RULES



How Should DFD Components Be Connected?

DFD components must be connected logically. Data stores should not connect directly to external entities, and external entities should communicate with data stores through processes.

The lesson also states that data flows should not cross one another for clarity.



Examples:

➡ **Process Connection** – Processes act as the point where data is handled between other components.

➡ **Data Store Connection** – Data should pass through a process before entering or leaving a data store.

➡ **Non-Crossing Flow** – Data flow lines should be arranged clearly instead of crossing each other.



LEVEL 1 DFD



What Does a Level 1 DFD Show?

A Level 1 DFD breaks the main system process into smaller subprocesses. It shows the processes, data flows between them, data stores, and external entities that interact with the subprocesses.



Examples:



Sub-process – A smaller activity created by breaking down the main process.



Data Flow – Shows how information moves between the subprocesses and other components.



Data Store – Shows where information is kept within the detailed process.



ELEMENTS OF A LEVEL 1 DFD

 What Elements Appear in a Level 1 DFD?

A Level 1 DFD contains several elements that work together to describe the detailed flow of data.

The lesson identifies processes, data stores, external entities, and data flows as important elements.

 Examples:

 Processes – Smaller tasks or functions, commonly numbered such as 1.0, 2.0, and 3.0.

 External Entities – People or systems outside the system boundary that interact with it.

 Data Flows – Arrows that show how information moves between components.



LEVEL 0 OR CONTEXT DFD



What Does a Context Diagram Show?

A Level 0 DFD, also called a Context Diagram, presents the entire system as one single main process.

It gives the simplest big-picture view by showing the system, its external entities, and the major inputs and outputs.



Examples:



Main System – Represents the entire system as one process.



External Entity – Represents a person or system interacting with the main system.



Input/Output – Shows major information entering or leaving the system.



LEVEL 1 DFD EXAMPLE



How Is a System Broken Down in Level 1?

A Level 1 DFD takes the single main process from Level 0 and divides it into smaller processes.

This allows us to see more clearly how information moves inside the system and how external entities interact with specific processes.



Examples:

-  **Process 1.0** – Represents one major subprocess.
-  **Process 2.0** – Represents another subprocess connected to the system flow.
-  **Data Flow** – Connects processes and shows the information being transferred.



LEVEL 2 DFD



How Does Level 2 Add More Detail?

A Level 2 DFD provides a more detailed breakdown of subprocesses from a Level 1 DFD.

The lesson's Login System illustration shows processes such as User Registration, Registration Validation, User Account Management, Verify Account Details, and User Account Confirmation.



Examples:



Registration Validation – A detailed subprocess that handles registration information.



User Account Management – A detailed process related to managing user account information.



Account Confirmation – A process that confirms verified account details.



DO'S OF DFD



What Connections Are Allowed in a DFD?

DFDs follow connection patterns that ensure data passes through processes correctly.

The lesson identifies several acceptable connections between entities, processes, and data stores.



Examples:

➡ Entity–Process–Entity – Data can move from an external entity through a process to another external entity.

➡ Data Store–Process–Data Store – Data can move between stores through a process.

➡ Entity–Process–Data Store – An entity can provide data to a process that stores the information.



DON'TS OF DFD



Which DFD Connections Should Be Avoided?

Some direct connections are not allowed because they bypass the process that should handle the data.

The lesson identifies Entity–Entity, Data Store–Data Store, Entity–Data Store, and Data Store–Entity as connections to avoid.



Examples:

➡ Entity–Entity – Two external entities should not directly exchange data in the DFD without a process.

➡ Data Store–Data Store – Two data stores should not directly exchange data without a process.

➡ Entity–Data Store – An external entity should not directly send data to a data store without a process.



FLOWCHART



What Is a Flowchart?

A Flowchart is a visual diagram that shows the step-by-step flow of activities or logic in a process.

It can show the sequence of tasks and decisions, making it useful for understanding how a process starts, what happens next, and where different conditions lead.



Examples:



Sequence – Shows the order in which activities happen.



Activity – Represents an action that must be performed.



Decision – Represents a condition that determines which path the process follows.



FLOWCHART COMPONENTS



What Symbols Are Used in a Flowchart?

Flowcharts use standard symbols to represent different types of actions and relationships.

The lesson shows symbols for start/end, arrows, input/output, processes, and decisions.

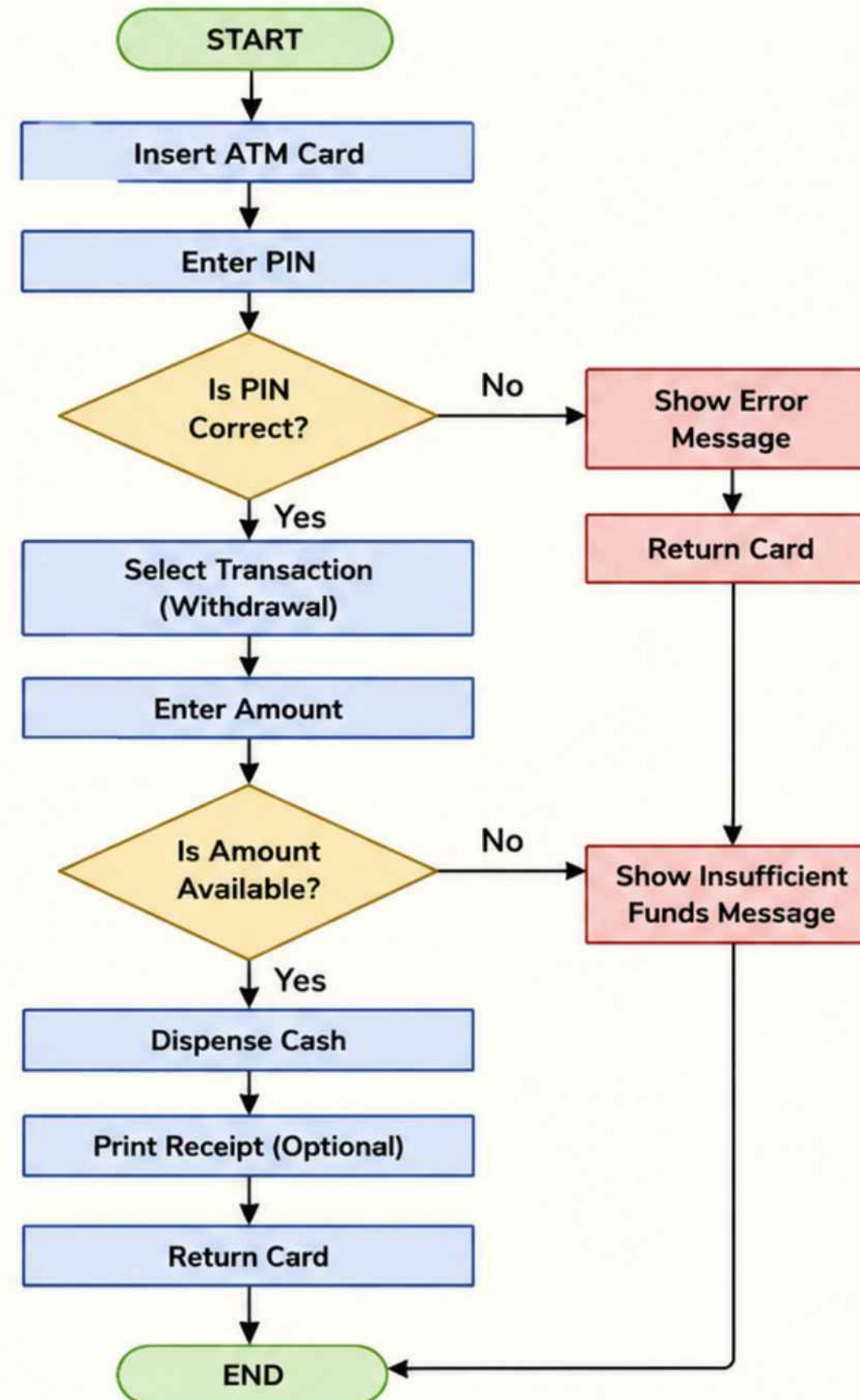


Examples:

-  **Start/End** – An oval represents where the process begins or ends.
-  **Process** – A rectangle represents an activity or process.
-  **Decision** – A diamond represents a condition that determines the next path.

EXAMPLE OF FLOWCHART

EXAMPLE: ATM CASH WITHDRAWAL PROCESS



SYMBOL	MEANING
	Start / End Indicates the beginning or end of the process.
	Activity (Process) Represents an action or operation.
	Decision Represents a condition that has two or more possible paths.
	Flowline Shows the direction of flow in the process.

FLOWCHART EXPLANATION

- The process starts when the user inserts the ATM card.
- The user enters their PIN.
- If the PIN is incorrect, an error message appears and the card is returned.
- If correct, the user selects withdrawal and enters amount.
- If the amount is not available, an insufficient funds message appears.
- If available, cash is dispensed, optional receipt is printed, and the card is returned.
- The process ends.

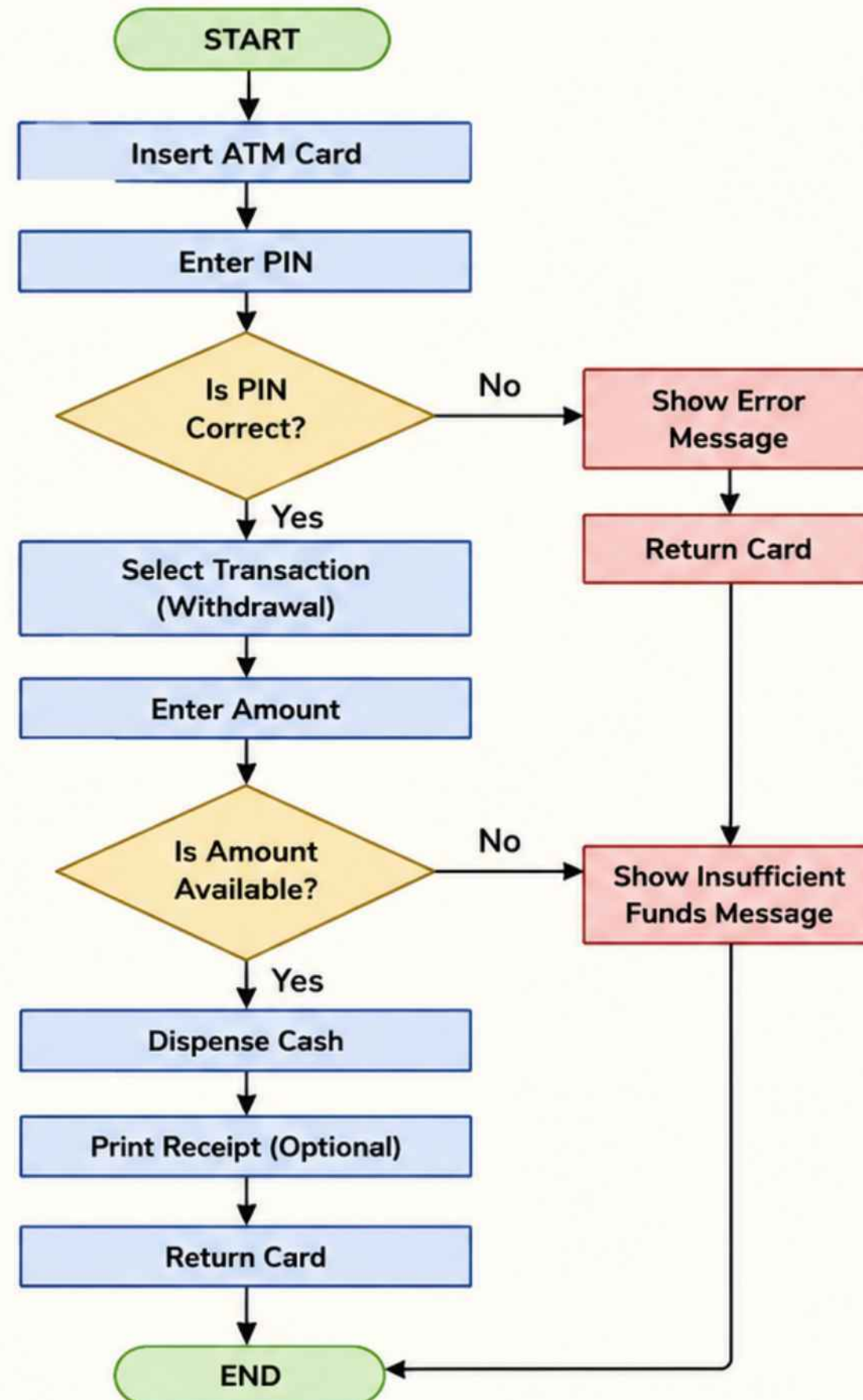


CONCLUSION:

A flowchart shows the sequence of activities and decisions in a process.

EXAMPLE OF FLOWCHART

EXAMPLE: ATM CASH WITHDRAWAL PROCESS



SYMBOL	MEANING
	Start / End Indicates the beginning or end of the process.
	Activity (Process) Represents an action or operation.
	Decision Represents a condition that has two or more possible paths.
	Flowline Shows the direction of flow in the process.

FLOWCHART EXPLANATION

- The process starts when the user inserts the ATM card.
- The user enters their PIN.
- If the PIN is incorrect, an error message appears and the card is returned.
- If correct, the user selects withdrawal and enters amount.
- If the amount is not available, an insufficient funds message appears.
- If available, cash is dispensed, optional receipt is printed, and the card is returned.
- The process ends.



CONCLUSION:

A flowchart shows the sequence of activities and decisions in a process.



FLOWCHART SAMPLE: ORDERING A BOOK



How Can a Flowchart Show an Online Ordering Process?

A flowchart can represent a more complex process with several activities and decisions.

The lesson's sample includes entering a website, checking registration, selecting a book, reviewing the order, entering shipping information, selecting payment, and placing the order.



Examples:



Registration – Determines whether the user is already registered or needs to sign up.

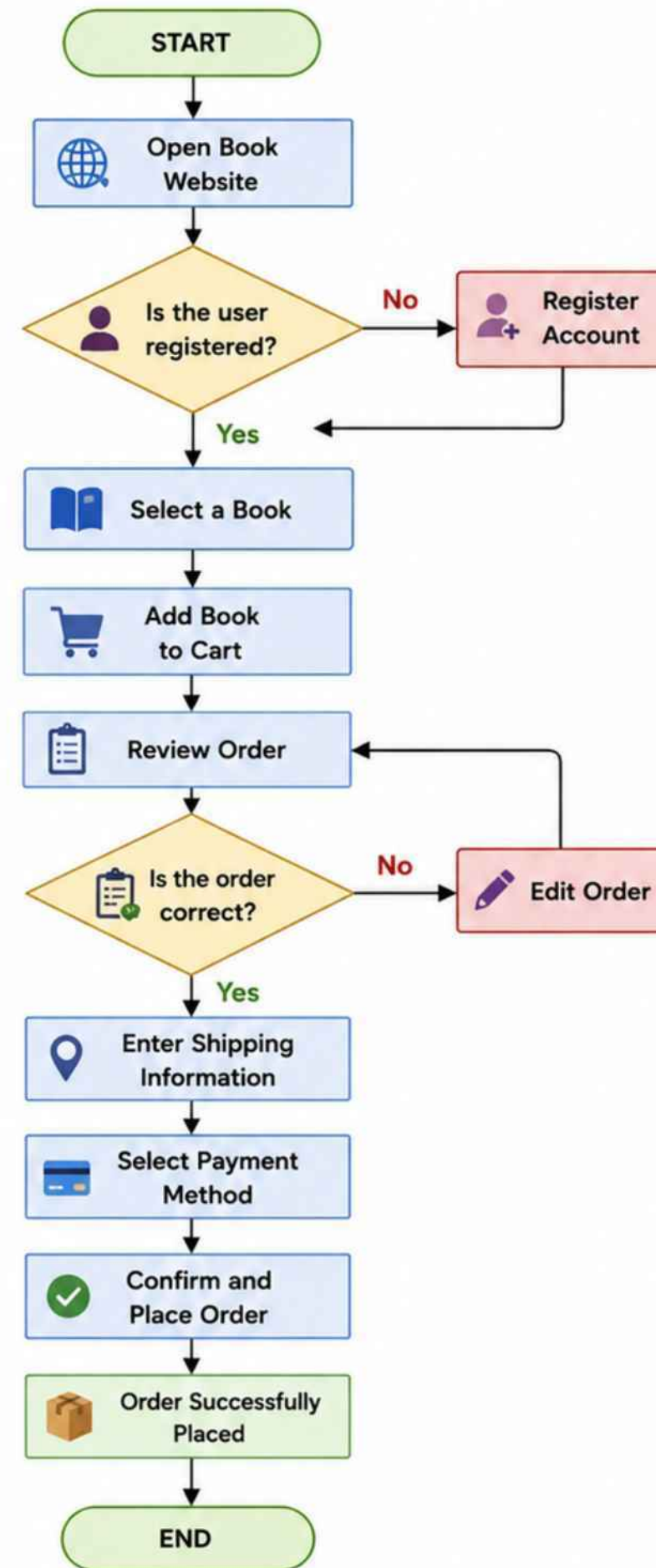


Book Selection – Allows the user to select the preferred book.



Order Confirmation – Checks whether the information is correct before placing the order.

FLOWCHART SAMPLE: ORDERING A BOOK



LEGEND (SYMBOLS)

	Terminator (Start / End) Indicates the beginning or end of the process.
	Process / Activity Represents an action or operation.
	Decision Represents a condition that has two or more possible paths.
	Flowline Shows the direction of flow.

FLOWCHART EXPLANATION

- 1 Open the book website.
- 2 Check if the user is registered.
 - If No → register a new account.
 - If Yes → continue to select a book.
- 3 Select the desired book.
- 4 Add the book to the cart.
- 5 Review the order.
- 6 Check if the order is correct.
 - If No → edit the order.
 - If Yes → continue.
- 7 Enter shipping information.
- 8 Select a payment method.
- 9 Confirm and place the order.
- 10 Order is successfully placed.
- 11 End of the process.

KEY POINTS

- ✓ Registration determines if the user is new or existing.
- ✓ Book selection allows the user to choose the preferred book.
- ✓ Order confirmation ensures all details are correct before placing the order.



UML DIAGRAMS



What Is UML?

UML (Unified Modeling Language) is a visual language used to design and describe systems.

It helps developers, designers, and stakeholders understand a system before it is built. UML can show system structure and behavior and is commonly used in object-oriented programming.



Examples:



UML – A visual language for modeling and describing systems.



System Structure – Shows how parts of a system are organized and related.



System Behavior – Shows how users or system elements interact and perform activities.



UML CLASS DIAGRAM



What Does a Class Diagram Show?

A UML Class Diagram illustrates the static structure of a system by showing classes, their attributes, operations, and relationships.

A class is shown as a rectangle divided into three sections: class name, attributes, and methods or operations.



Examples:



Class Name – Appears at the top section of the class rectangle and identifies the class.



Attributes – Represent the data or properties associated with a class.



Methods – Represent functions or operations that the class can perform.



UML CLASS DIAGRAM: BANKING SYSTEM



How Is a Class Diagram Used in a Banking System?

A banking system can use a class diagram to show the different classes and how they are related.

The lesson includes a banking system class diagram containing classes such as Customer, Bank, Account, ATM, and ATM Transactions, with attributes, methods, and relationships.



Examples:



Customer – Represents a customer in the banking system and contains customer-related information.



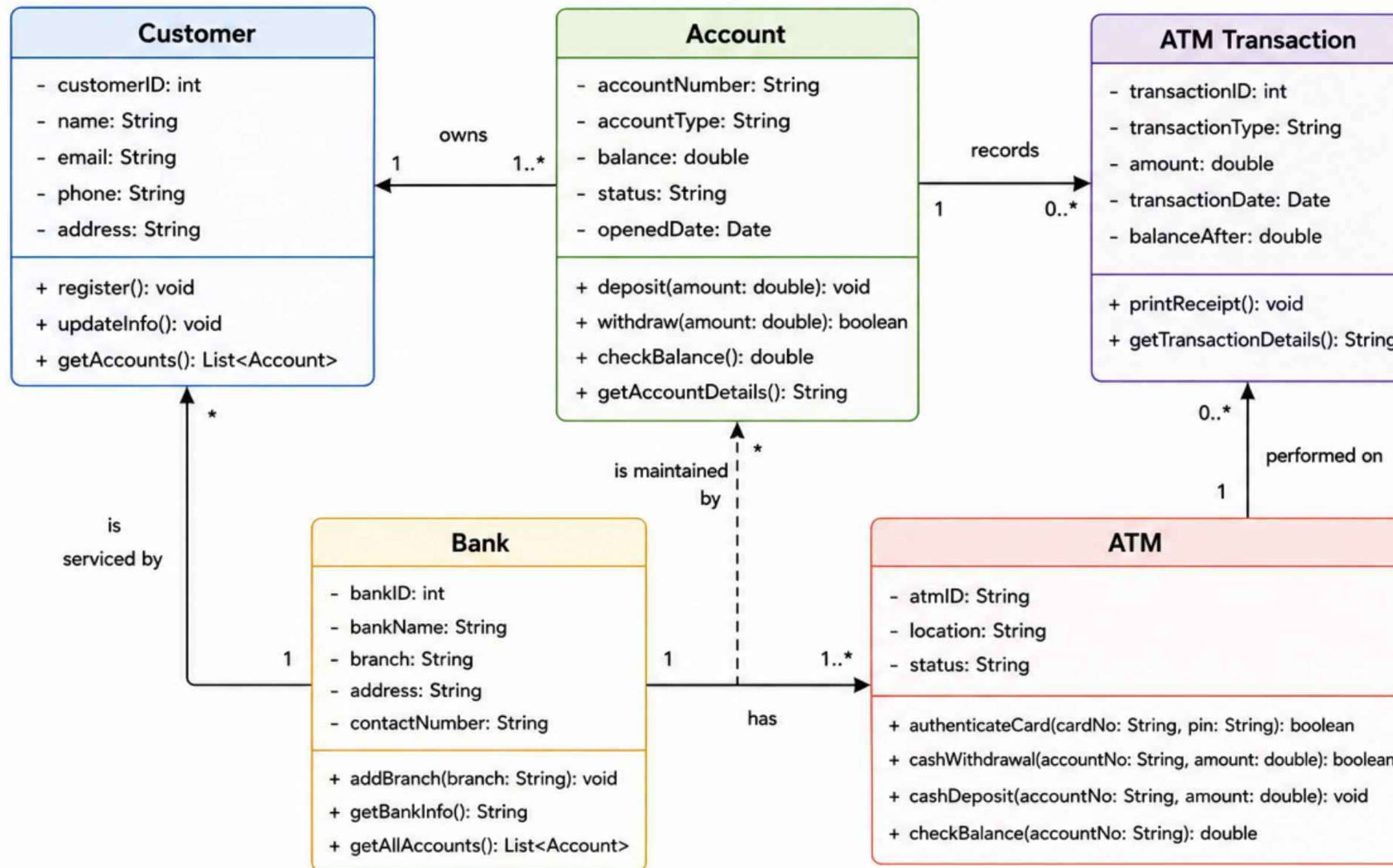
Account – Represents an account and contains information such as account number and balance.



ATM – Represents the ATM system component that can perform operations such as withdrawal, deposit, and balance checking.

UML CLASS DIAGRAM: BANKING SYSTEM

• Example: Banking System •



CLASS DIAGRAM GUIDE

ClassName

Class Name

- attribute1: DataType
- attribute2: DataType
- attribute3: DataType

Attributes
(properties)

+ method1(): ReturnType
+ method2(): ReturnType

Methods
(operations)

RELATIONSHIP LEGEND

- Association (uses / interacts with)
- Navigable Association (one-way relationship)
- Dependency (depends on)

MULTIPLICITY GUIDE

- 1 Exactly one
- 0..1 Zero or one
- 1..* One or more
- 0..* Zero or more
- * Many (unspecified)



KEY TAKEAWAY

- A class diagram shows the static structure of the system.
- It includes classes, attributes, methods, and relationships.
- This diagram helps in understanding how the classes in a banking system are related and interact with each other.



UML CLASS DIAGRAM RELATIONSHIPS



How Can Classes Be Related?

UML class diagrams use different relationship types to describe how classes are connected.

The lesson identifies association, multiplicity, inheritance/generalization, aggregation, and composition as relationship concepts.



Examples:



Association – Shows a relationship between classes using a solid line.



Inheritance – Shows a generalization relationship using a line with a hollow triangle arrow.



Composition – Shows a strong whole-part relationship using a filled diamond.



UML USE CASE DIAGRAM



What Is a Use Case Diagram?

A UML Use Case Diagram is a visual representation of how users or actors interact with a system.

It provides a high-level view of system behavior from the user's perspective and helps stakeholders, developers, and analysts understand how the system is intended to operate.



Examples:



Actor – Represents a user or external participant interacting with the system.



Use Case – Represents a function or service that the system provides.



System Boundary – Shows the boundary of the system and contains its use cases.

UML USE CASE DIAGRAM

A UML Use Case Diagram shows how **actors (users)** interact with a system to achieve specific goals.

EXAMPLES



Actor

Represents a user or external participant interacting with the system.



Use Case

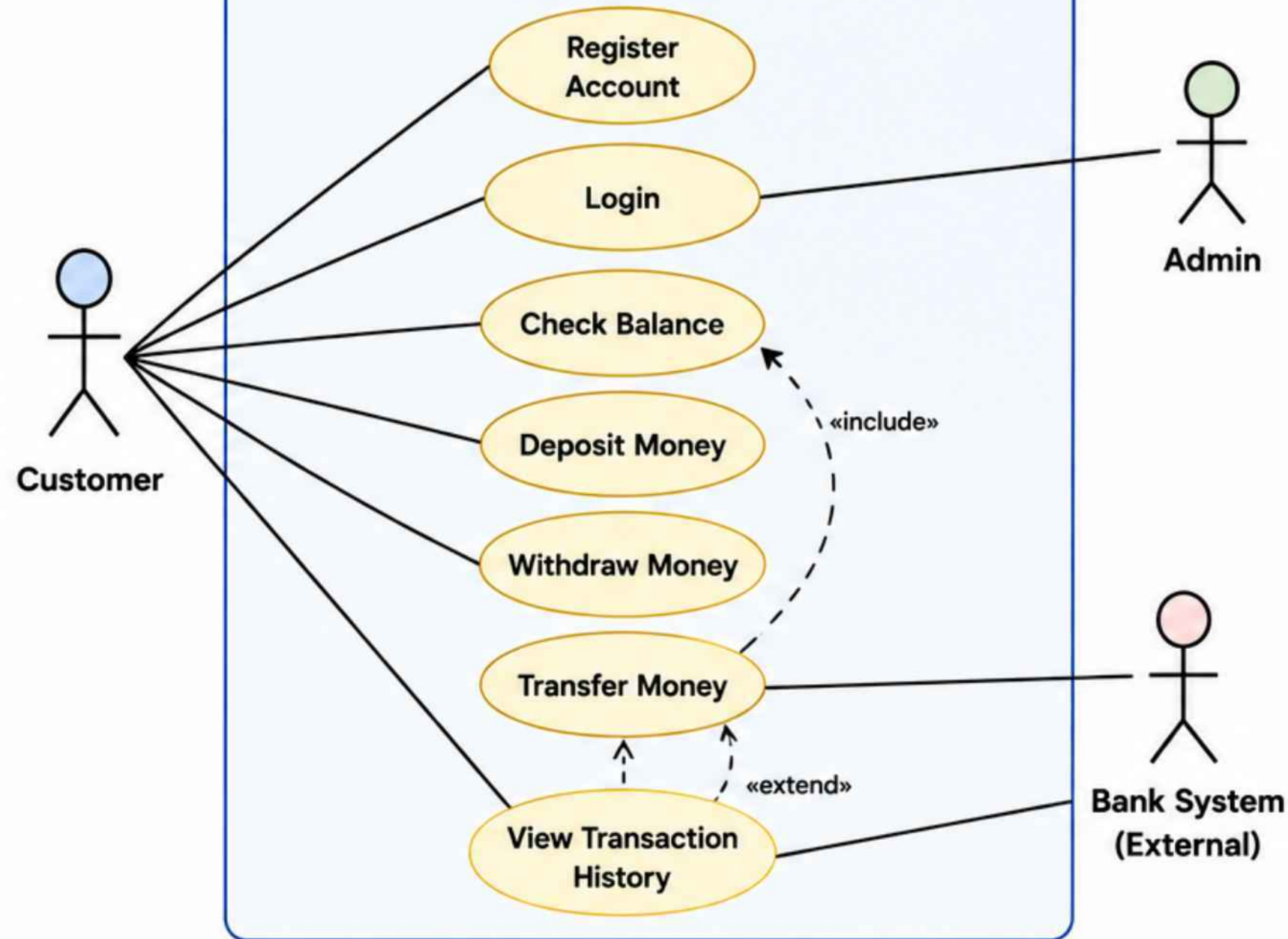
Represents a function or service that the system provides.



System Boundary

Shows the boundary of the system and contains its use cases.

ONLINE BANKING SYSTEM



SYMBOLS GUIDE



Actor

A user or external participant.



Use Case

A function or service provided by the system.



System Boundary

Defines the scope and boundary of the system.

RELATIONSHIP GUIDE



Association

Communication between actor and use case.



Include

Mandatory inclusion of another use case.



Extend

Optional extension of a use case.

KEY TAKEAWAY



- Use case diagrams focus on WHAT the system does, not HOW it does it.
- They help identify the system's functions and the actors who use them.
- They are useful in requirements gathering and system analysis.



NOTE

This diagram represents the high-level interaction between users and the Online Banking System.



USE CASE DIAGRAM NOTATIONS



What Symbols Are Used in a Use Case Diagram?

Use Case Diagrams use standard visual symbols to represent actors, system functions, and the system boundary.

These notations make it easier to identify who interacts with the system and which functions are available.



Examples:

➡ Actor – Shown as a stick-figure symbol representing a user or external participant.

➡ Use Case – Shown as an oval representing a system function.

➡ System Boundary – Shown as a rectangle surrounding the system's use cases.

USE CASE DIAGRAM NOTATIONS



Use Case Diagrams use standard visual symbols to represent actors, system functions, and the system boundary.

These notations make it easier to identify who interacts with the system and which functions are available.

NOTATION	SYMBOL	DESCRIPTION	EXAMPLE
	Actor	- Represents a user or external participant who interacts with the system. - Placed outside the system boundary.	 Customer
	Use Case	- Represents a function or service that the system provides. - Describes what the system does from the user's perspective.	 Login
	System Boundary	- Defines the scope and boundary of the system. - Contains all the use cases of the system.	 Online Banking System Login
	Association	- Shows communication between an actor and a use case. - A solid line connects the actor to the use case.	
	Include	- Indicates that a use case always includes another use case. - Drawn as a dashed line with an open arrow and the keyword «include».	
	Extend	- Indicates that a use case adds optional or conditional behavior to another use case. - Drawn as a dashed line with an open arrow and the keyword «extend».	



KEY TAKEAWAY

These notations help in visualizing the system's functions and how users interact with it. They are essential in requirements gathering, system analysis, and communication.

QUICK REMINDER

- ✓ Actor – Stick Figure
- ✓ Use Case – Oval

- ✓ System Boundary – Rectangle
- ✓ Lines – Show Relationships



USE CASE: ONLINE BANKING SYSTEM



How Can a Use Case Diagram Show Banking Interaction?

In an online banking system, a customer can interact with different system functions.

The lesson shows an association between the Customer actor and the Transfer Funds use case, representing communication between the actor and the system function.



Examples:



Customer – The actor who uses the banking system.



Transfer Funds – A system function available to the customer.

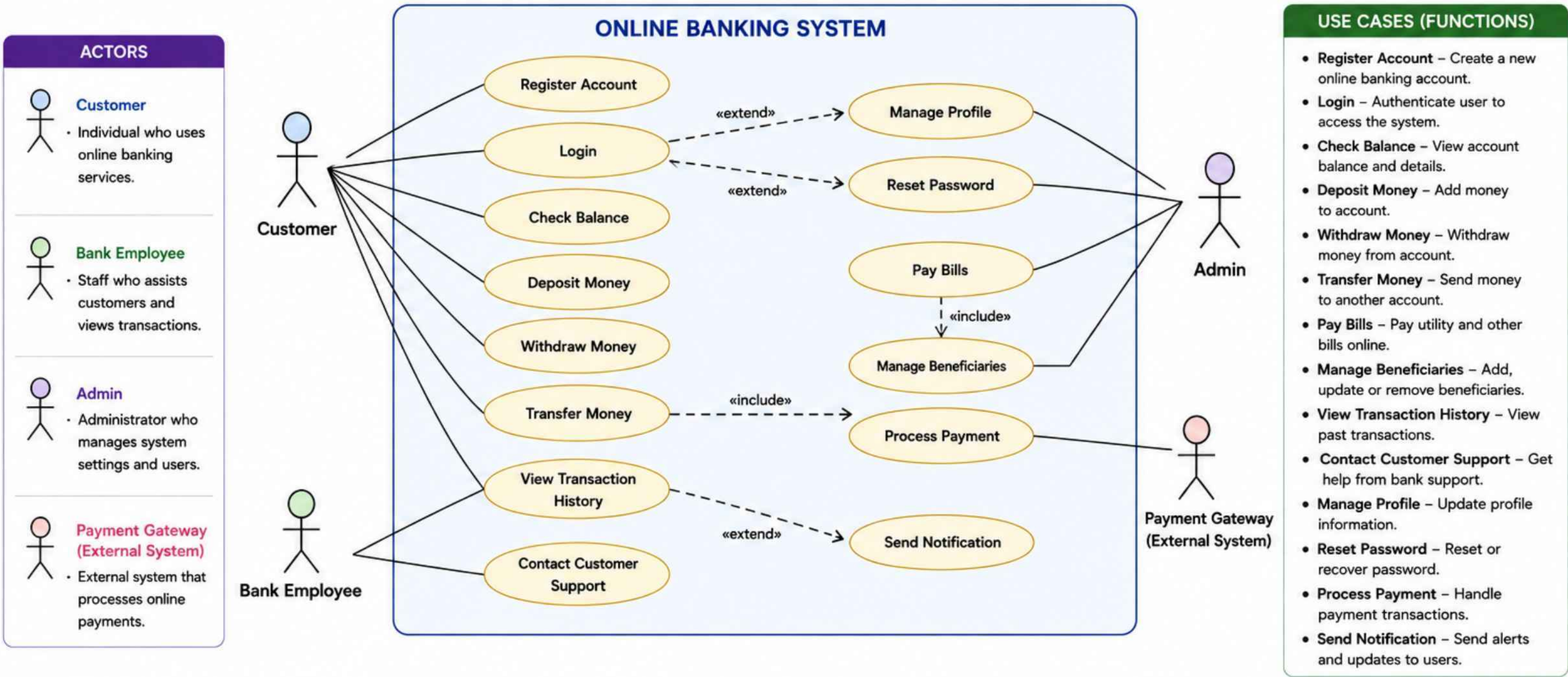


Association – Shows the interaction between the customer and the use case.

USE CASE DIAGRAM: ONLINE BANKING SYSTEM



A Use Case Diagram shows how users (actors) interact with the Online Banking System to perform tasks and achieve their goals.



RELATIONSHIP LEGEND		
	Association	Communication between an actor and a use case.
	«include»	Required use case – always included in the process.
	«extend»	Optional use case – adds additional behavior to the base use case.

KEY TAKEAWAY

- Customers can perform banking transactions online.
- Some functions are required (include), such as checking balance before transferring money.
- Some functions are optional (extend), such as viewing transaction history or resetting password.
- External systems like Payment Gateway interact with the system to complete specific tasks.

NOTES

- Use case diagrams focus on **WHAT** the system does, not **HOW** it does it.
- This diagram represents the high-level functions and interactions in the Online Banking System.
- It helps in requirements gathering and system analysis.



USE CASE: SOCIAL MEDIA POSTING



What Does “Include” Mean in a Use Case Diagram?

The include relationship shows that one use case includes the functionality of another use case.

The lesson's social media example shows Compose Post including Add Image, meaning the included functionality is part of the main use case.



Examples:



Compose Post – A main social media function for creating a post.



Add Image – A function that can be included as part of composing a post.



Include – Indicates that one use case uses the functionality of another use case.

USE CASE: SOCIAL MEDIA POSTING

The «include» relationship shows that one use case includes the functionality of another use case. The included use case is always executed as part of the main use case.

EXAMPLE EXPLANATION

● Compose Post

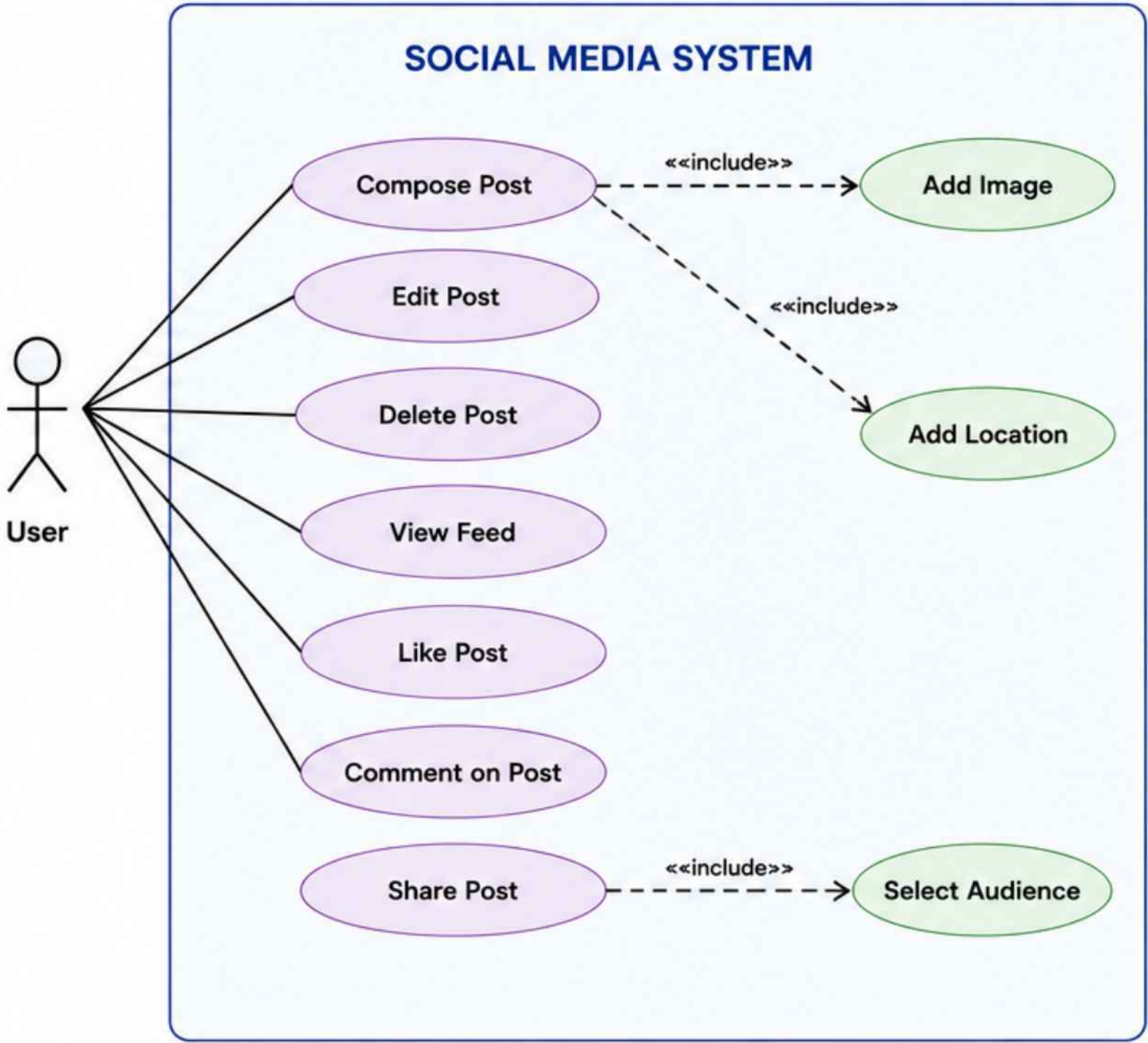
The main social media function for creating a post.

● Add Image

A function that can be included as part of composing a post.

--> Include

Indicates that one use case uses the functionality of another use case.



USE CASES (FUNCTIONS)

- Compose Post – Create a new post.
- Edit Post – Modify an existing post.
- Delete Post – Remove a post.
- View Feed – View posts from friends or followed pages.
- Like Post – Like a post.
- Comment on Post – Add a comment to a post.
- Share Post – Share a post to your timeline.
- Add Image – Add an image to the post.
- Add Location – Attach location to the post.
- Select Audience – Choose who can see the post.

RELATIONSHIP GUIDE

—	Association Shows communication between an actor and a use case.
- - - - ->	«include» Indicates that the included use case is always executed as part of the main use case.



KEY TAKEAWAY

- The «include» relationship is used when the main use case always needs another use case to complete its process.
- In Social Media Posting, to Compose a Post, the system will always perform Add Image (if image is added), Add Location (if location is attached), and Select Audience (to choose who can see the post).

Example:

User wants to create a post with picture, location, and choose audience.

The system will:

Compose Post → Add Image → Add Location → Select Audience



USE CASE: FLIGHT BOOKING SYSTEM



What Does “Extend” Mean in a Use Case Diagram?

The extend relationship shows optional or additional behavior that can occur under certain conditions.

The lesson's flight booking example shows Select Seat extending Book Flight, meaning seat selection can add behavior to the flight booking process.



Examples:



Book Flight – The main use case for booking a flight.



Select Seat – An additional function that can be performed when the user wants to choose a specific seat.



Extend – Represents additional behavior added to another use case under certain conditions.

USE CASE: FLIGHT BOOKING SYSTEM



The «extend» relationship shows optional or additional behavior that can occur under certain conditions. The extending use case adds functionality to the base use case when needed.

EXAMPLE EXPLANATION

● Book Flight

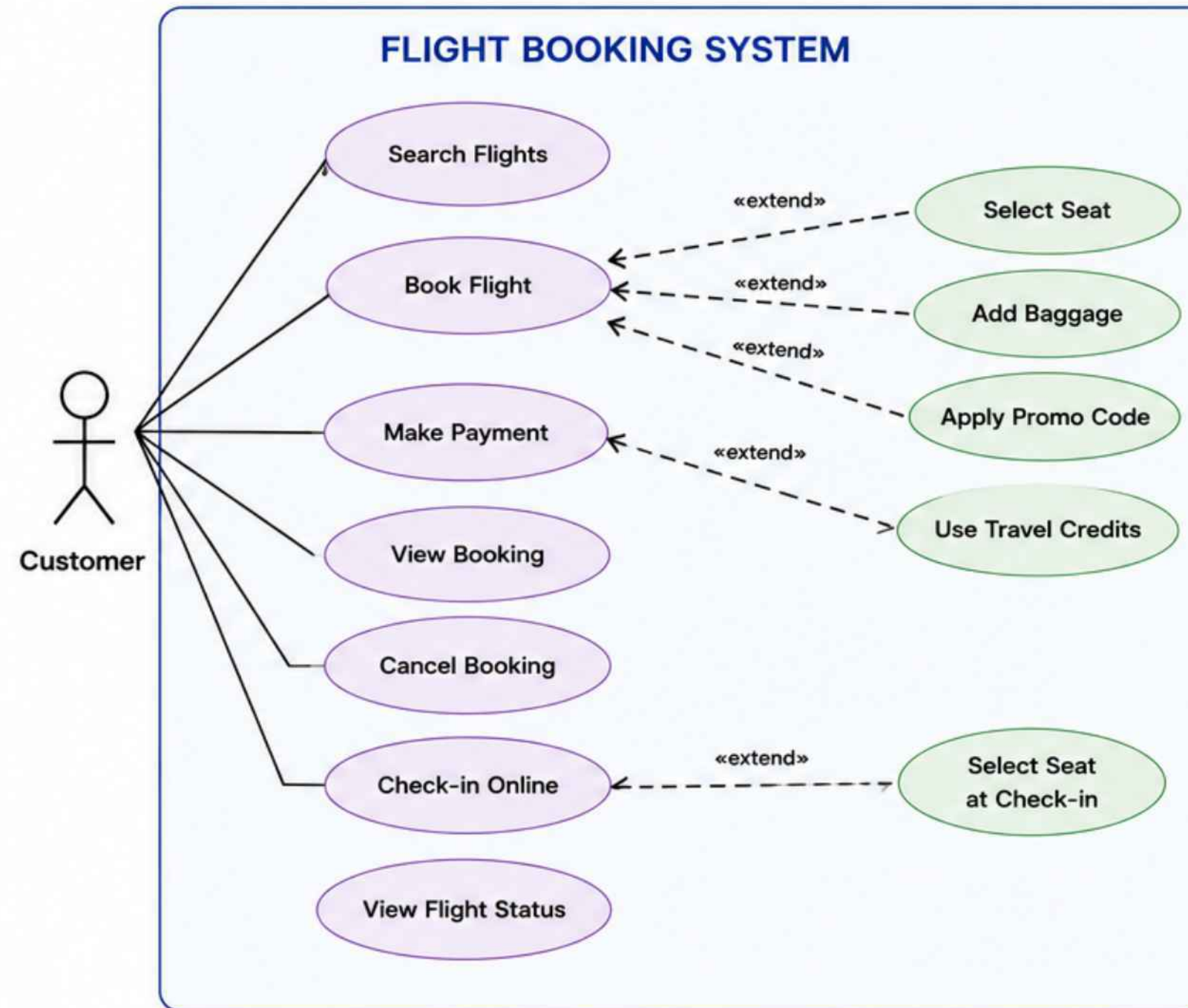
The main use case for booking a flight.

● Select Seat

An additional function that can be performed when the user wants to choose a specific seat.

--> Extend

Indicates optional or conditional behavior that extends another use case.



USE CASES (FUNCTIONS)

MAIN USE CASES

- Search Flights – Search available flights.
- Book Flight – Book a selected flight.
- Make Payment – Pay for the booking.
- View Booking – View booking details.
- Cancel Booking – Cancel a booking.
- Check-in Online – Check in for a flight.
- View Flight Status – Check status of flight.

EXTENDING USE CASES (OPTIONAL)

- Select Seat – Choose a specific seat.
- Add Baggage – Add extra baggage.
- Apply Promo Code – Use a discount code.
- Use Travel Credits – Pay using credits.
- Select Seat at Check-in – Choose seat during online check-in.

RELATIONSHIP GUIDE

	Association Shows communication between an actor and a use case.
	«extend» Shows optional/conditional behavior that adds functionality to the base use case when a condition is met.



KEY TAKEAWAY

- The «extend» relationship is used when the additional behavior is not always needed.
- The base use case is complete on its own, but the extending use case adds extra functionality only when a specific condition is true.

Example Scenario:

A customer wants to book a flight. The system will perform the main process: Search Flights → Book Flight → Make Payment. If the customer wants extra services, they can optionally:

Select Seat / Add Baggage / Apply Promo Code / Use Travel Credits.

These are extensions to the main booking process.

USE CASE: VEHICLE RENTAL SYSTEM



Generalization shows that specific use cases can be specialized versions of a more general use case. The specific use cases inherit the behavior of the general use case.

EXAMPLE EXPLANATION

Rent Vehicle

The general use case representing vehicle rental.

Rent Car

A specialized version of renting a vehicle for cars.

Rent Bike

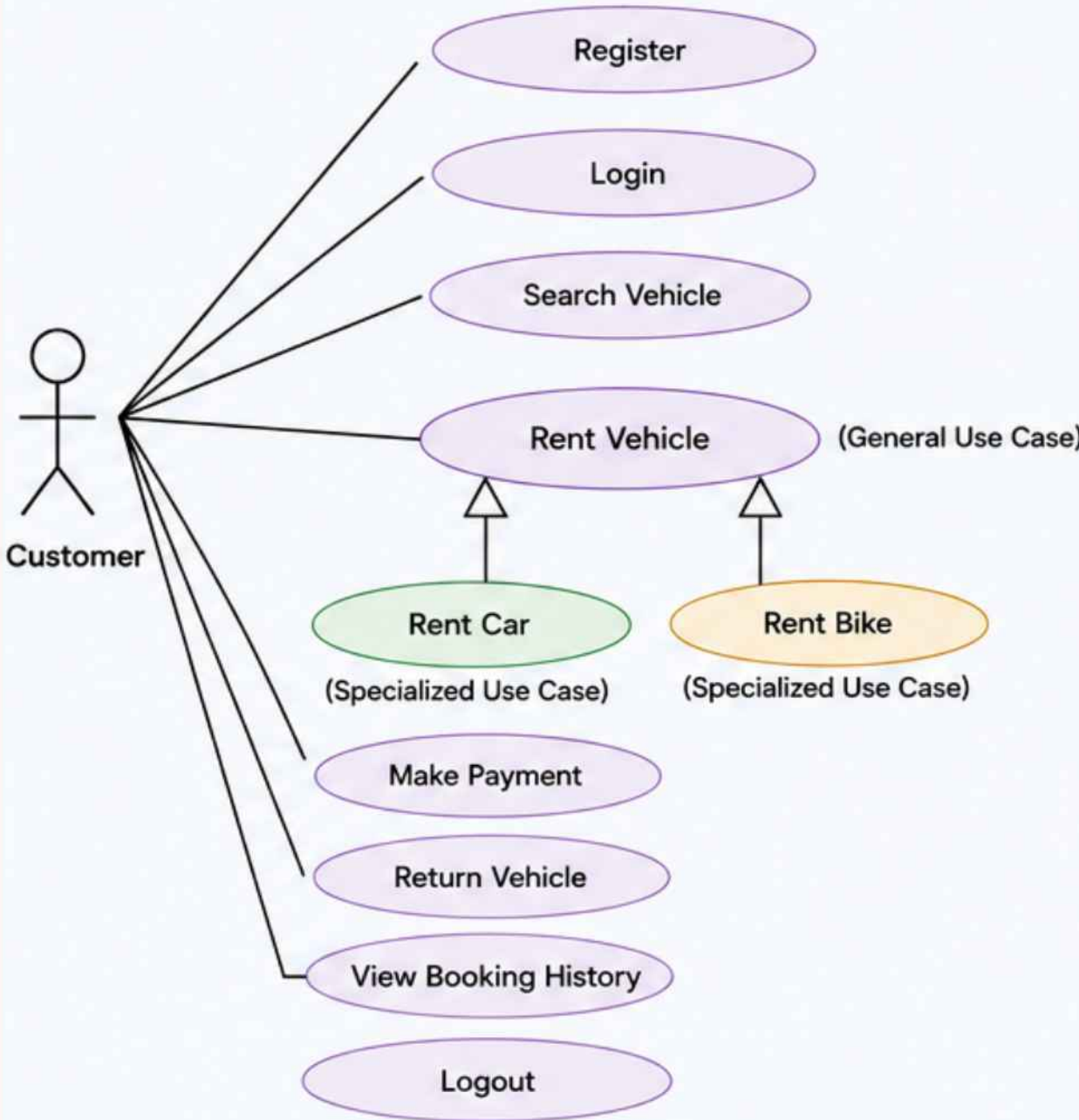
Another specialized version of renting a vehicle for bikes.



Generalization

Indicates that the child use cases are specialized versions of the parent use case.

VEHICLE RENTAL SYSTEM



USE CASES (FUNCTIONS)

GENERAL USE CASE

- Rent Vehicle – The general process of renting any type of vehicle.

SPECIALIZED USE CASES

- Rent Car – Rent a car.
- Rent Bike – Rent a bike.

OTHER USE CASES

- Register – Create a new account.
- Login – Access the system.
- Search Vehicle – Search available vehicles.
- Make Payment – Pay for the rental.
- Return Vehicle – Return the rented vehicle.
- View Booking History – View past rentals.
- Logout – Sign out from the system.

RELATIONSHIP LEGEND

—	Association Shows communication between an actor and a use case.
↑	Generalization Shows that a specific use case is a specialized version of a general use case.



KEY TAKEAWAY

- Generalization represents an “is-a” relationship.
- Specialized use cases (children) inherit the behavior of the general use case (parent).
- It helps avoid duplication and makes the diagram easier to understand and maintain.

Example Scenario:

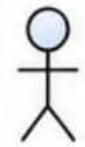
A customer wants to rent a vehicle. The system has a general process “Rent Vehicle”. Depending on the customer’s choice, the process can be specialized into: **Rent Car** or **Rent Bike**. Both follow the general rental flow but have their own specific details.

USE CASE: LIBRARY MANAGEMENT SYSTEM



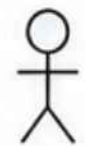
A library management system contains several use cases that represent different functions performed by users (actors). These use cases work together to provide complete library services.

ACTORS (USERS)



Librarian

An actor who manages books, members and library operations.



Member

An actor who uses library services such as borrowing and returning books.

SYMBOLS USED



Actor

Represents a user or external entity.



Use Case

Represents a function or service provided by the system.



Association

Shows the connection between actor and use case.



Include (<<include>>)

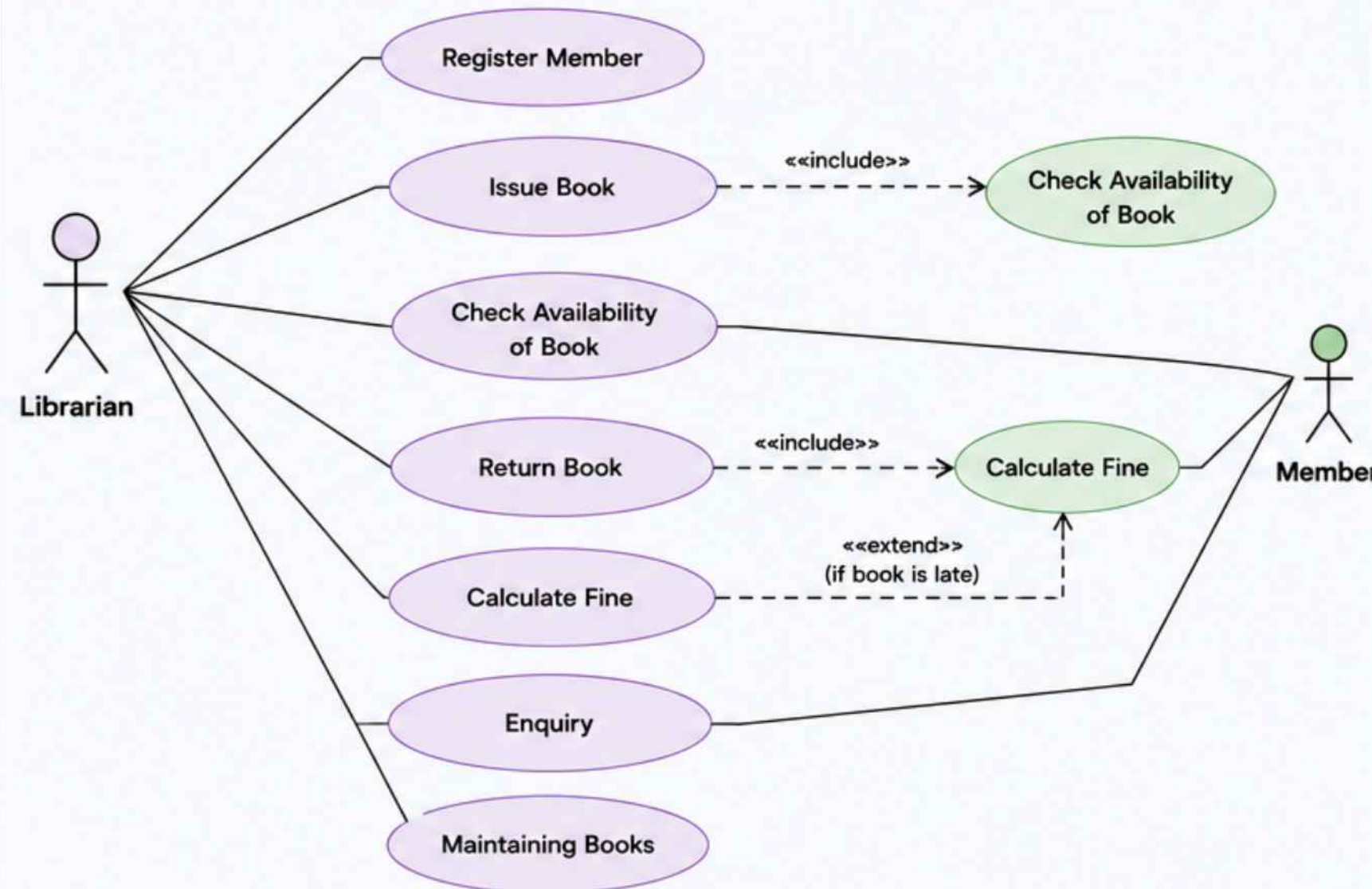
Required use case that is always performed.



Extend (<<extend>>)

Optional use case that adds behavior under certain condition.

LIBRARY MANAGEMENT SYSTEM



USE CASES (LIBRARY FUNCTIONS)

- 1 **Register Member** – Add new member into the library system.
- 2 **Issue Book** – Allow member to borrow a book.
- 3 **Check Availability of Book** – Check if the book is available in the library.
- 4 **Return Book** – Accept returned book from member.
- 5 **Calculate Fine** – Calculate fine if the book is returned late.
- 6 **Enquiry** – Search or ask information about books or members.
- 7 **Maintaining Books** – Add, update or remove book information.

RELATIONSHIP GUIDE

—	Association Shows communication between actor and use case.
- - - - ->	Include (<<include>>) The use case is always required as part of the process.
- - - - ->	Extend (<<extend>>) The use case is optional and happens only if a condition is true.



KEY TAKEAWAY

- Multiple use cases work together to achieve the goals of the system.
- Actors interact with different use cases based on their role.
- Include is used for mandatory actions.
- Extend is used for optional or conditional actions.

EXAMPLE SCENARIO

A member wants to borrow a book. The system will first Issue Book. The system must Check Availability of Book (<<include>>). After reading, the member will Return Book. If the book is returned late, the system will Calculate Fine (<<extend>>). The librarian can register members, maintain books, handle enquiries and manage all library operations.

**THANK
YOU**

